

Sistema Operativo

Un sistema operativo è un software che si interfaccia con i componenti hardware della macchina e fornisce servizi fondamentali per l'esecuzione di programmi applicativi. Un calcolatore moderno presenta componenti hardware complesse come uno o più *processori*, *memoria* principale, *dischi* e *unità* flash, periferiche I/O.

L'Hardware supporta due modalità ben distinte:

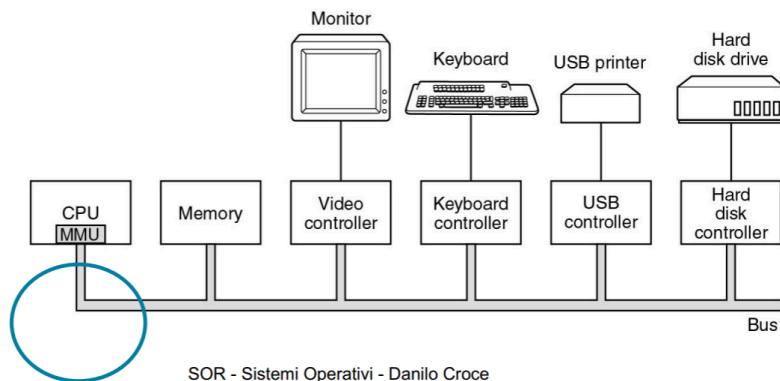
- Modalità **utente** (quella a cui hanno diretto accesso anche le applicazioni);
- Modalità **kernel** (ha accesso il SO che ha visione totale delle componenti hardware).

Il SO mette a disposizione l'hardware attraverso le *chiamate di sistema*.

Vi sono 3 prospettive per descrivere il SO:

1. Idea di *astrazione*: il SO si pone tra HW e SW;
2. Visione *top-down*: il SO fornisce astrazioni ai programmi applicativi;
3. Visione *bottom-up*: il SO gestisce parti di un sistema complesso, e fornisce un'allocazione ordinata e controllata delle risorse.

Un SO moderno gestisce più programmi in esecuzione e più utenti, motivo per il quale necessita di un'allocazione ordinata e controllata delle risorse.



Processore

Central Processing Unit - **CPU**, esegue istruzioni dalla memoria secondo il ciclo base di *fetch-decode-execute*. Le CPU eseguono un set specifico di istruzioni tramite registri interni che memorizzano dati importanti e risultati. Tra i più importanti:

- *Program Counter*: Indica l'istruzione successiva;
- *Stack Pointer*: Punta alla cima dello stack in memoria, che contiene frame di procedure con parametri e variabili locali;

- **Program Status Word**: Contiene info sullo stato del programma, fondamentale per chiamate di sistema e I/O.

Multiplexing : il SO esegue programmi in modo efficiente.

Pipeline : esegue in parallelo istruzioni che possono essere eseguite a livello circuitale. Anche in caso di un'istruzione condizionale, la pipeline esegue anche l'operazione successiva pur di non fermarsi dall'eseguire istruzioni.

Più di un processore : più processori fisici o logici, *multithreading*.

Dispositivi I/O

Due componenti principali: **Controller** (più semplice da usare per il SO); **Dispositivo** (interfaccia elementare ma difficile da pilotare).

Ogni controller ha bisogno di un *driver* per il SO, con cui interagisce attraverso le porte di I/O con istruzioni di tipo **IN/OUT** e mappatura in memoria.

Per eseguire l'I/O:

- Il processo esegue la chiamata di sistema;
- Il kernel effettua una chiamata al driver;
- Il driver avvia l'I/O.

Ed interroga il dispositivo per vedere se è disponibile, o gli chiede di generare un interrupt quando ha terminato.

DMA

Direct Memory Access, dispositivo HW speciale che consente ai componenti di accedere direttamente alla memoria del computer **senza** coinvolgere la CPU.

- Migliora l'efficienza ed aumenta le prestazioni I/O ad alta velocità
- Viene utilizzato per il trasferimento veloce di dati tra memoria e dispositivi periferici
- Riduce il carico sulla CPU, consentendole di concentrarsi su altri compiti

Buses

Implementazione di bus supplementari per gestire l'aumento del traffico dati. *La CPU comunica con la memoria attraverso un bus veloce DDR4, con una periferica grafica esterna sul bus PCIe, e con tutti gli altri dispositivi attraverso un hub su un bus DMI.*

Avvio del sistema

La memoria della scheda madre contiene il firmware (**BIOS**, *Basic Input Output System*). Dopo l'accensione, la CPU esegue il BIOS che inizializza RAM e risorse, ed avvia il **boot loader**, con cui viene infine caricato il SO.

syscall

Il SO offre funzionalità attraverso le **chiamate di sistema**, che implementano servizi.

I processi sono astrazioni a livello utente per eseguire un programma per conto dell'utilizzatore. Ogni processo ha il proprio spazio di indirizzamento ed i dati coinvolti nell'elaborazione vengono recuperati e memorizzati in file (che persistono rispetto ai processi).

Le **syscall** sono in pratica il meccanismo con cui un'applicazione chiede servizi al SO.

Processo

Processo - Programma in esecuzione, associato ad uno spazio di indirizzi e ad un insieme di risorse. Può essere pensato come un contenitore di tutte le informazioni necessarie per l'esecuzione di un programma.

Le info sui processi sono conservate nella tabella dei processi del SO. Un processo può essere *creato*, *terminato*, messo in *pausa* e *ripreso*. Un processo può creare un altro processo (detto *figlio*) creando una gerarchia di processi.

I processi sono di proprietà di un utente, identificato da un UID.

File

File - Astrazione di un dispositivo di memorizzazione, su cui è possibile leggere/scrivere dati fornendo una posizione ed una quantità di dati da trasferire. Sono organizzati in *directory* che conservano un identificatore per ogni file che contengono.

File e directory formano una gerarchia, che inizia dalla directory principale (**root**) e si sviluppa in percorsi, che possono essere *assoluti* (partono da root) o *relativi* (partono dalla directory di lavoro).

I file sono "protetti" da tuple a 3 bit per **proprietario**, **gruppo** e **altri utenti**. Le tuple contengono un bit (r)ead, (w)rite ed un bit e(x)ecute

Struttura di un SO

Monolitico

Il programma principale invoca le chiamate di sistema richieste. Il kernel è un blocco monolitico con:

- Procedure di servizio che eseguono le chiamate di sistema
- Procedure di utility che aiutano ad implementare le procedure di servizio

Virtualizzazione

Le **Macchine Virtuali (VM)** permettono l'esecuzione di più SO su un unico HW fisico, simulando un ambiente separato per ciascuno.

Vantaggi:

- **Isolamento**: Ogni VM opera indipendentemente
- **Flessibilità**: Possibilità di eseguire diversi SO simultaneamente
- **Gestione semplificata**: Facilita gestione e manutenzione separando multiprogrammazione e risorse HW

Container

I **Container** mantengono la stessa idea delle VM di isolamento logico, ma **senza un OS per istanza**. Essi fanno in modo di venire incontro alle esigenze moderne di **ridurre l'overhead**, di ottenere un **avvio rapidissimo** e di mantenere la **portabilità dell'ambiente**.

I Container permettono di eseguire più istanze isolate di applicazioni e servizi sopra lo **stesso kernel** del SO host. Ogni container condivide kernel e librerie di base con l'host, ma mantiene filesystem, librerie e configurazioni proprie. Tuttavia portano anch'essi degli svantaggi:

- Non è possibile eseguire SO diversi dall'host
- L'isolamento è meno forte rispetto alle VM

Kernel

Esistono tre principali approcci per la progettazione del kernel di un SO:

- **Exokernel**: Idea di separare il controllo delle risorse dalla macchina estesa. Non emula l'HW, ma fornisce solo una condivisione sicura delle risorse a basso livello. Ogni VM a livello utente esegue il suo SO, ma è limitata ad utilizzare solo

le risorse assegnate, quindi elimina la necessità di mappature complesse, concentrandosi solo su quale VM ha accesso a quali risorse.

- **Unikernel**: Sistemi minimi progettati per eseguire una singola applicazione su una VM. Contengono solo la funzionalità necessaria per supportare l'applicazione specifica.
- **Microkernel**: Organizza le *service procedures* che vengono eseguite in modo separato. I processi di sistema comunicano attraverso il passaggio di messaggi, come le chiamate di sistema. Ogni processo del SO può fare solo ciò che è necessario per svolgere la propria task, e la compromissione non influisce sul resto del SO, ma il passaggio di messaggi è più lento di una chiamata di funzione.